HARNESS LAB · MÓDULO 4 · AULA 4.1

# Quem decide o pronto *não é o modelo*

O "pronto" do modelo é um palpite plausível. Pronto é um comando que volta passa ou falha, rodado fora da palavra dele, e um juiz de fora com a caderneta na mão.

`exit 0` `qual-comando.sh` `LOOP_TEST_CMD` `## Pronto` `@agent-juiz` `model: sonnet` `/goal` `Stop`

VOCÊ SAI COM

A Prova do seu loop (o comando de teste), verde de partida e escrita no `CLAUDE.md` junto com a definição de pronto, e um juiz de fora (subagente no Sonnet, só leitura) que confere a entrega item por item contra a sua caderneta.

**Material de apoio** · Lucas Souza · Beer and Code

# 1 A aula em *uma página*

A aula parte do trilho que você já tem (o `CLAUDE.md`, a documentação viva e os hooks) e põe **avaliações que não dependem do agente**, feitas por fora. Antes de construir, pergunte: essa feature é verificável? Como garanto que ele chegou?

**Quem decide o pronto não é o modelo sozinho: é você.** A especificação traz a definição de pronto, o checklist do que precisa existir, e validações automáticas marcam esse checklist.

### Plausível não é certo

O pós-treinamento com feedback humano ensina o modelo a ser plausível. Na dúvida, ele chuta: admitir incerteza é a pior nota. O "pronto" dele é um palpite.

> *"language models are optimized to be good test-takers, and guessing when uncertain improves test performance."*
>
> Kalai et al. · Why Language Models Hallucinate · arXiv 2509.04664, p. 1

O modo operador já tem formas embutidas, como o `/goal` e o `/code-review`. A aula vai além: uma **prova final**, com portões que dizem "deu certo" ou "volte e corrija".

### Ele burla quando não fecha

Mesmo o modelo mais forte. Sem os insumos para chegar do outro lado, ele tenta até achar um atalho para o "completed", quando devia parar e dizer que não tem o que fazer.

### O que conta como prova

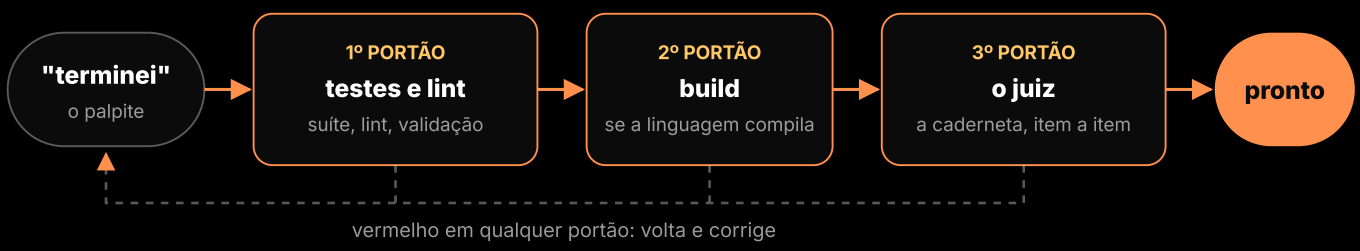
Teste, a prova em código determinístico. Build, se a linguagem compila. Linter, gate de qualidade. Script que compara a saída com a esperada. Teste bom cobre regra de negócio, não 100% de cobertura.

### O que você já tem

O quality gate com lint e teste (2.2), o `Stop` que não deixa passar vermelho (2.4) e o `test-runner` da lição de casa (3.4), que diz qual teste falhou.

### O juiz de fora

O inspetor com a caderneta: cadastro de usuários? Tem rota, controller, model. Disparar e-mail? Tem o job. Confere pelo código e pode validar no navegador, com Playwright (o E2E da 4.4).




O juiz, o terceiro portão, **não precisa ser maior que o construtor**: avaliar é muito mais simples que construir. O Lucas escreve no Opus 5.5 e avalia no Sonnet 5. O que dá errado é a feature gigante e mal quebrada; por isso o planejamento vai ser quebrado em boas janelas de contexto. Os portões custam tempo: é o preço de pisar do outro lado com a feature concluída.

## 2 Indo *além da aula*

Daqui em diante, **Prova** é o comando de teste que o loop roda fora da sessão, e **juiz** é o agente de fora com a caderneta: o gate 2 e o gate 3 do `ralph.sh` (o gate 1 vê se a sessão escreveu algo).

**ALÉM DA AULA · A PROVA SÓ LÊ O EXIT CODE**

O loop roda o comando com `bash -c` e só olha o código de saída: 0 é verde, o resto é vermelho. Três jeitos de enganar a Prova sem querer: um pipe (`pest | tail -20` devolve o exit do `tail`, 0, com teste vermelho), um `|| true` herdado do CI e o script de teste que o `npm init` escreve (`echo "Error: no test specified" && exit 1`), vermelho para sempre. No Python, o pytest sai com 5 quando não coleta nenhum teste.

Fonte: `ralph.sh`, `gate2_tests_pass`; pytest, "Exit code 5: No tests were collected". Os três primeiros, testados no `teste/verificar.sh`.

**ALÉM DA AULA · BUILD E LINT NO MESMO PORTÃO**

A Prova roda um comando só. Para o build (ou o typecheck) e o lint contarem, encadeie com `&&`: o primeiro que falha corta a fila, e o exit é o dele. No Node, a cadeia mora no script de teste (a detecção segue achando `npm test`): `"test": "tsc --noEmit && eslint . && vitest run"`. No Laravel com Sail, vai no `LOOP_TEST_CMD`: `vendor/bin/sail pint --test && vendor/bin/sail test`. A mesma cadeia vai na linha `Testes:` do `CLAUDE.md`.

Fonte: best-practices, "When it's done": "Tests, typecheck, a `Stop` hook, CI"; Laravel Pint, `--test`; o `pint` do Sail. Testado no `teste/verificar.sh`.

**ALÉM DA AULA · UM COMANDO, TRÊS PORTÕES**

Três peças já rodam teste, cada uma com a sua regra. O `Stop` do quality gate (2.2) roda só os testes dos arquivos editados, no Sail se houver `vendor/bin/sail` e um compose. O `prova-no-stop.sh` (2.4) roda `vendor/bin/pest` no host. A Prova do loop roda a suíte inteira, com `vendor/bin/sail test` se houver `artisan` e `laravel/sail` no `composer.json`, mesmo sem compose. Verde num, vermelho no outro. Use um comando só: a linha `Testes:` do `CLAUDE.md` (o `test-runner` da 3.4 a usa), o do hook e, se a detecção errar, o `LOOP_TEST_CMD`.

Fonte: `quality-gate.sh` (2.2), `prova-no-stop.sh` (2.4), `test-runner.md` (3.4), `ralph.sh` e `loop.sh` (5.1).

**ALÉM DA AULA · VERDE DE PARTIDA**

A Prova roda a suíte inteira em toda fase e em todo ciclo. Um teste vermelho de antes do loop reprova até a fase que só cria uma migration e vira a causa de cada ciclo de correção. Por isso a meta é `exit 0` hoje, antes de qualquer feature. Executável ausente ou Sail parado abortam o loop antes da primeira sessão; sem comando, o `ralph.sh` puro desliga a Prova com um aviso, e o `loop.sh` do curso nem começa.

Fonte: `ralph.sh` (`resolve_test_cmd`, `check_sail_running`, `gate2_tests_pass`); pré-voo do `loop.sh` (5.1): "Sem Prova, não tem loop".

### ALÉM DA AULA · /GOAL E /CODE-REVIEW PERGUNTAM A UM MODELO

Os dois estão na coluna "não determinística" da tabela da doc. O avaliador do `/goal` roda no modelo pequeno e rápido e julga só a conversa: "It doesn't run commands or read files independently". Sem a saída do teste na conversa, ele não tem o que ler. Ponha na condição o comando e o que não pode mudar: `/goal npm test sai com 0 e nenhum arquivo de teste foi alterado`. O `/code-review` caça bug no diff, num subagente que nunca viu a conversa, e não confere a sua lista. Para conferir a lista, a doc sugere um pedido seu: revisar o diff contra a SPEC, requisito por requisito, "and skip style".

Fonte: code.claude.com/docs/en/goal, "Write an effective condition" e "How evaluation works"; best-practices, "Review against your spec"; code-review-local. Pedidos em `modelos/pedir-a-prova.md`.

### ALÉM DA AULA · POR QUE SEPARAR O JUIZ, E QUANTO CUSTA AVALIAR

"Separating the agent doing the work from the agent judging it proves to be a strong lever", escreve a Anthropic: quem fez tende a elogiar o próprio trabalho, e é mais fácil deixar um avaliador cético do que tornar crítico quem gerou. No mesmo artigo, a primeira construção de um app levou 2 h 7 min e US$ 71,08; a avaliação dela, 8,8 min e US$ 3,24. Avaliar custa uma fração de construir; se o Sonnet acerta no seu projeto, o passo 8 mostra. O autor avisa: sem ajuste, o avaliador acha o problema e se convence de que não é grave; daí o "Na dúvida, INCOMPLETE" do `juiz.md`. O viés de o Claude favorecer o Claude é pequeno no modelo atual: 0,07 ponto em 10. Cuidado: o modelo pedido na chamada vence o `model:` do arquivo.

Fonte: Anthropic Engineering, Harness design for long-running application development (24/03/2026), tabela "Agent & Phase" e "Claude is a poor QA agent"; System Card Claude Opus 5.5, §6.5.3, p. 127; sub-agents, "Choose a model".

### ALÉM DA AULA · A PORTA DE SAÍDA PARA A TAREFA QUE NÃO FECHA

Dê ao agente um jeito honesto de parar. O guia de prompting da Anthropic tem a frase: "If the task is unreasonable or infeasible, or if any of the tests are incorrect, please inform me rather than working around them." A versão em português é o bloco `## Pronto` do passo 6. É pedido, não trava: diminui o atalho, não impede. Na 4.3, o critério ambíguo vira critério binário, e o convite ao atalho diminui de novo.

**3 a 6×** mais tentativas de atalho quando a tarefa é impossível do que quando é possível; cerca de 80% delas é entregar incompleto sabendo. Taxa por episódio de treino, medida por classificador, não "trapaça no seu projeto".

Fonte: System Card Claude Opus 5.5 (22/09/2026), §6.2.2, p. 101; platform.claude.com, Prompting best practices, "Avoid focusing on passing tests and hardcoding".

# 3 Instale e faça *no seu repositório*

## 1 Baixe os materiais do curso (uma vez só).

Terminal · na sua pasta pessoal — shell

```
cd ~
git clone https://github.com/beerandcodeteam/harness-lab-materiais
```

Já baixou antes? Atualize:

Terminal · em qualquer pasta — shell

```
git -C ~/harness-lab-materiais pull
```

## 2 Instale a aula no seu projeto.

Cria `.claude/scripts/qual-comando.sh` e `.claude/agents/juiz.md`. Não depende de aula anterior. Arquivo com o mesmo nome que já é seu fica intacto, e o da aula vai para `.harness-lab-novo`.

Terminal · na raiz do seu projeto — shell

```
~/harness-lab-materiais/instalar.sh 4.1
```

## 3 Descubra o comando de prova.

A mesma escolha do pré-voo do `loop.sh` do Módulo 5: o `LOOP_TEST_CMD`, se você passar; senão a detecção do `ralph.sh` (Sail, `composer test`, `php artisan test`, `npm test`, `pytest`, `go test ./...`, `cargo test`). Só lê arquivos; nunca o `.env`.

Terminal · na raiz do seu projeto — shell

```
bash .claude/scripts/qual-comando.sh
```

| SAÍDA | QUER DIZER | FAÇA |
|---|---|---|
| `exit 0` | Comando e origem resolvidos | Siga para o passo 4 |
| `exit 3 · (nenhum)` | Projeto sem teste, ou stack fora da lista (Rails, Java, .NET) | Sem teste: o pedido abaixo. Com teste: `LOOP_TEST_CMD="bundle exec rspec"` na frente |
| `exit 4` | O executável do comando não existe ou não está no PATH | `composer install`, instale a ferramenta ou use `LOOP_TEST_CMD` |
| `exit 5` | Sail detectado, containers parados | `vendor/bin/sail up -d`; sem Sail, `LOOP_TEST_CMD` (abaixo) |
| `exit 6` | `RALPH_TEST_CMD` exportada no seu shell | `unset RALPH_TEST_CMD`: o `loop.sh` recusa essa variável |

No starter kit do Laravel o comum é o `exit 5`: ele traz `laravel/sail` no `composer.json`, e a detecção escolhe o Sail mesmo que você não use. Troque o comando com a variável na frente, só naquela linha. No Módulo 5, o mesmo `LOOP_TEST_CMD` vai na frente do `loop.sh`, que recusa `RALPH_TEST_CMD` exportada e não roda sem comando de teste.

Na raiz do projeto · Laravel sem Sail — laravel

```
LOOP_TEST_CMD="./vendor/bin/pest" bash .claude/scripts/qual-comando.sh
# Rails: LOOP_TEST_CMD="bundle exec rspec" na frente
```

Projeto sem teste nenhum? Peça ao Claude:

> Este projeto não tem testes. Instale o runner de testes padrão da stack, crie um teste que passa de verdade (não um assert true) e configure o script de teste no manifest do projeto (composer.json, package.json ou pyproject.toml). Me mostre o comando e a saída.

### 4 Deixe verde de partida.

Roda o comando e devolve o exit code dele; o esperado é `exit: 0`. Usou `LOOP_TEST_CMD` no passo 3? Ponha na frente aqui também. Vermelho: conserte agora, antes de qualquer feature (seção 2, "Verde de partida").

Terminal · na raiz do seu projeto — shell

```
bash .claude/scripts/qual-comando.sh --rodar
```

### 5 Veja a Prova reprovar.

No editor, troque um valor esperado num teste que passa (`toBe(1)` por `toBe(2)`, `assertEquals(1, ...)` por 2), salve e rode. O esperado é a falha, `exit: 1` e `vermelho: 1`. Desfaça a troca (no editor ou com `git restore` no arquivo do teste) e rode de novo: `exit: 0`, e o `git status --short` sem o teste.

Terminal · na raiz do seu projeto — shell

```
bash .claude/scripts/qual-comando.sh --rodar || echo "vermelho: $?"
git status --short
```

### 6 Escreva a Prova e o pronto no CLAUDE.md.

"The single most useful line in your `CLAUDE.md` is the command that runs it" (best practices); o `test-runner` da 3.4 lê essa linha. O `## Pronto` é a definição de pronto da aula, com a porta de saída da seção 2. Peça ao Claude (também em `modelos/pedir-a-prova.md`):

> Rode bash .claude/scripts/qual-comando.sh e grave o comando que ele mostrar no CLAUDE.md, seção ## Comandos, numa linha que começa com "Testes: ". Depois crie a seção ## Pronto com estes três itens, sem mudar o texto: "Pronto é o comando de testes acima saindo com 0. Mostre o comando e a saída, não um resumo.", "Teste que parece errado: me avise. Não edite nem apague teste para ele passar." e "Tarefa impossível ou faltando informação: pare e diga o que falta, em vez de contornar."

Usou `LOOP_TEST_CMD`? Dê o seu comando no lugar do script. Confira: `Testes:` igual à `comando:` do passo 3, e o `## Pronto` no arquivo.

Terminal · na raiz do seu projeto — shell

```
grep -n "Testes:" CLAUDE.md
grep -n "## Pronto" CLAUDE.md
```

7

**Confira o juiz e chame ele.**

O esperado é "Validation passed", sem "with warnings". Abra o Claude Code na raiz; se a pasta `.claude/agents/` acabou de nascer, reinicie: ele só vigia as pastas que existiam no início da sessão.

Terminal · na raiz do seu projeto — shell

```
claude plugin validate .claude/agents
```

Dentro do Claude Code — sessão

```
> @agent-juiz confira os critérios do FEATURE-ALVO.md contra o código
```

A resposta: uma linha por critério (`AC-01: DONE · arquivo:linha` ou `INCOMPLETE · o que falta`) e o `VEREDITO`. Com a feature-alvo ainda por construir, o certo hoje é `NÃO PRONTO`. Sem lista no pedido nem `FEATURE-ALVO.md` na raiz, ele responde `SEM CRITÉRIO` e para.

8

**Opcional: tire a prova do Sonnet.**

O mesmo juiz no Opus. Compare as linhas `AC-NN` e o `VEREDITO`: onde os dois discordam, leia o código você mesmo. Não há placar pronto; o resultado vale para o seu projeto e a sua caderneta.

Dentro do Claude Code — sessão

```
> rode o @agent-juiz de novo com model opus e compare os vereditos
```

9

**Opcional: o juiz que roda o teste de cada critério.**

Variante com Bash travado pelo hook `bash-so-permitido.sh` da 3.4: só `./vendor/bin/sail artisan test --filter Nome` (com espaço: o hook bloqueia `--filter=`), `route:list` e `grep`. Depois da 4.4, roda o E2E do critério do mesmo jeito (o Pest browser usa o Playwright). Sem Sail, `php artisan test --filter` e `php artisan route:list`; outra stack, `npx vitest run`, `pytest -q -k`: troque no frontmatter e no corpo. A primeira linha instala a 3.4, que traz o hook; o `cp` troca o seu `juiz.md`. Depois, abra o Claude Code e aceite a confiança da pasta, senão o hook do subagente não roda.

Terminal · na raiz do seu projeto — shell

```
~/harness-lab-materiais/instalar.sh 3.4
A=$(echo ~/harness-lab-materiais/modulo-4-prova/aula-4.1-*)
cp "$A/modelos/juiz-que-roda-teste.md" .claude/agents/juiz.md
```

### O juiz *instalado*

.claude/agents/juiz.md — instalado pelo instalar.sh · trecho resumido · não copie do PDF

```
---
name: juiz
description: Confere uma entrega contra a definição de pronto, lendo o código real. [...]
tools: Read, Grep, Glob
model: sonnet
---
Você é o juiz. Não escreve, não edita e não roda nada. O resumo de quem construiu não é prova.
## 1. A caderneta: a lista do pedido; senão o arquivo citado (SPEC.md, PHASES.md); senão os
"Critérios de aceite" do FEATURE-ALVO.md. Sem lista: SEM CRITÉRIO, e para.
## 2. Como julgar: ignore "terminei". Cada item: o código e o teste que o confere; tela, o E2E.
Ausente, placeholder, teste faltando, pulado ou pendente: INCOMPLETE. Na dúvida, INCOMPLETE.
## 3. A resposta
AC-01: DONE · app/Http/Controllers/RegisterController.php:42 · tests/Feature/CadastroTest.php:18
AC-02: INCOMPLETE · não há teste para e-mail com letras maiúsculas
VEREDITO: NÃO PRONTO
```

## 4 Arquivos que *reforçam a aula*

`.claude/agents/juiz.md`

O inspetor da aula: outro contexto, Sonnet, só leitura, "na dúvida, INCOMPLETE". O juiz do loop, na mão.

No repositório: modulo-4-prova/aula-4.1-quem-decide-se-esta-pronto/projeto/

`modelos/claude-md-prova.md`

A linha `Testes:` por stack, a cadeia com build e lint e o `## Pronto` do passo 6, com o porquê.

No repositório: modulo-4-prova/aula-4.1-quem-decide-se-esta-pronto/modelos/

`modelos/rules/testes.md`

A guideline que a aula cita: um teste por regra de negócio, com o nome da regra, não 100% de cobertura.

No repositório: modulo-3-regra/aula-3.2-guidelines-do-seu-projeto/modelos/rules/

`.claude/agents/test-runner.md`

A lição de casa da 3.4. Com a linha `Testes:`, roda o mesmo comando que o loop usa como Prova.

No repositório: modulo-3-regra/aula-3.4-subagente-quando-usar/projeto/

## 5 Erros *comuns*

| SINTOMA | CAUSA | CORREÇÃO |
|---|---|---|
| **`exit 5`, e você nem usa Sail** | O starter kit traz `laravel/sail` no `composer.json` | `vendor/bin/sail up -d`, ou `LOOP_TEST_CMD="./vendor/bin/pest"` aqui e no `loop.sh` |
| **`exit 6`, `RALPH_TEST_CMD` exportada** | Vence a detecção em qualquer pasta; o `loop.sh` recusa | `unset RALPH_TEST_CMD`; use `LOOP_TEST_CMD` por projeto |

| SINTOMA | CAUSA | CORREÇÃO |
|---|---|---|
| **`npm test` sempre vermelho** | O script do `npm init`: "Error: no test specified" | Troque pelo runner de verdade (`vitest run`) |
| **Teste vermelho e a Prova diz `exit: 0`** | Pipe ou `\|\| true`: vale o exit do último | Sem pipe; para somar build e lint, `&&` |
| **O juiz responde `SEM CRITÉRIO`** | Sem lista no pedido nem `FEATURE-ALVO.md` na raiz | Critérios no pedido, ou o `FEATURE-ALVO.md` (aula 1.5) |
| **Variante bloqueada em `--filter=`** | O hook da 3.4 só aceita a forma com espaço | Use `--filter Nome`; a 3.1 ensina `--filter=`, troque no seu `CLAUDE.md` |
| **Pus `Bash` no juiz e ele perdeu `Grep` e `Glob`** | Subagente com `Bash` fica sem os dois (Linux, macOS, WSL) | Só `Read, Grep, Glob`; ou `Bash, Read` com `grep` no hook |

## 6 Você sai *com*

- [ ] `bash .claude/scripts/qual-comando.sh` mostra o comando e a origem (detectado ou `LOOP_TEST_CMD`), e você sabe por que é esse.
- [ ] `--rodar` dá `exit: 0` (verde de partida) e voltou vermelho com um teste quebrado de propósito.
- [ ] O `CLAUDE.md` tem `Testes:` com esse comando e o `## Pronto`: evidência, teste errado se avisa, tarefa impossível para.
- [ ] `claude plugin validate .claude/agents` passa sem warning, e o `@agent-juiz` devolve uma linha por critério e o `VEREDITO`.
- [ ] `qual-comando.sh`, `juiz.md` e `CLAUDE.md` no commit: o time recebe a mesma Prova e o mesmo juiz.

PROMETIDO NA AULA

- ✓ **"Nós iremos dar para ele definições de pronto"**: o `## Pronto` do passo 6 (p. 6), o juiz (passo 7) e a 4.3.
- ✓ **"O /goal ou um /code-review"**: o que cada um vê (p. 4); condições em `modelos/pedir-a-prova.md`.
- ✓ **"Isso vai ser muito mitigado"** (a tarefa impossível): a porta de saída (p. 4), o `## Pronto` e a 4.3.
- ✓ **"Fazer bons testes é cobrir regra de negócio"**: `rules/testes.md` da 3.2 (seção 4) e as aulas 4.2 e 4.3.
- ✓ **"A lição de casa [...] o agente TestRunner"**: roda a Prova pela linha `Testes:` do passo 6 (p. 6); `modulo-3-regra/aula-3.4-subagente-quando-usar/projeto/.claude/agents/test-runner.md`.
- ✓ **"Eu gosto muito de colocar um juiz"**: passos 2 e 7 (p. 5 e 7), arquivo na p. 8; `modulo-4-prova/aula-4.1-quem-decide-se-esta-pronto/projeto/.claude/agents/juiz.md`.
- ✓ **"Utilizando um Playwright"**: aula 4.4. Depois dela, a variante do passo 9 (p. 7) roda o E2E do critério; o juiz só leitura lê o E2E e a saída que vier no pedido.
- ✓ **"A gente vai fazer uma quebra do planejamento"**: aula 4.3, `/bc-harness:plan` (SPEC, PLAN, PHASES).
- ✓ **"Eu utilizo o Sonnet em cima da escrita do Opus 5.5"**: `model: sonnet` (p. 8); o porquê e o custo (p. 4).
- ✓ **"Se quiser teimar, eu consigo te provar"**: passo 8 (p. 7), o juiz no Opus, `AC-NN` por `AC-NN`.

## 7 Glossário

| | |
|---|---|
| **Pronto** | A Prova com `exit 0` e a caderneta conferida pelo juiz; não a palavra do modelo. |
| **Prova** | O comando que volta passa ou falha. No loop, o detectado ou o `LOOP_TEST_CMD`. |
| **Exit code** | O número com que o comando termina: 0 é verde, o resto é vermelho. |
| **Gate (portão)** | Checagem que a entrega passa ou volta. No `ralph.sh`, a Prova é o gate 2; o juiz, o 3. |
| **Juiz** | Agente de fora, que não construiu, conferindo a entrega contra a caderneta. |
| **Caderneta** | A definição de pronto em lista: os critérios de aceite do `FEATURE-ALVO.md` ou da SPEC. |
| **Reward hacking** | O atalho para "passar" em vez de resolver: teste editado, valor fixo, entrega incompleta. |

## 8 Para *ir além*

- Best practices: "Give Claude a check to run" (a escada do pedido ao hook `Stop`) e "Review against your spec".
- Keep Claude working toward a goal: como o avaliador do `/goal` decide e o que ele não vê.
- Harness design for long-running application development: gerador e avaliador separados, e o custo de cada fase.
- Why Language Models Hallucinate: por que o treino e as avaliações premiam o chute confiante.
- System Card Claude Opus 5.5: §6.2, reward hacking e tarefa impossível; §6.5.3, o viés do juiz.

**Próxima: *Aula 4.2 · O teste verde que mente***

A Prova também pode mentir. Todos os testes passavam, e a integração estava quebrada.